Lab 3: Synchronization: Symbol Timing Recovery in Narrowband Channels

Summary

In this lab you will consider the problem of symbol timing recovery also known as symbol synchronization. Timing recovery is one of several synchronization tasks; others will be considered in future labs.

The wireless communication channel is not well modeled by simple additive white Gaussian noise. A more realistic channel model also includes attenuation, phase shifts, and propagation delays. Perhaps the simplest channel model is known as the frequency flat channel. The frequency flat channel creates the received signal

$$z(t) \quad = \quad \alpha e^{j\phi} x(t - \tau_d) + v(t), \tag{1}$$

where α is an attenuation, ϕ is a phase shift, and τ_d is the delay.

The objective of this lab is to correct for the delay caused by τ_d in discrete-time. The approach will be to determine an amount of delay $\hat{k}$ and then to delay the filtered received signal by $\hat{k}$ prior to downsampling. This will modified the receiver processing as illustrated in Figure 1.

Two algorithms will be implemented for symbol synchronization in this lab: the maximum energy method and the Early Late gate algorithm. The maximum energy method attempts to find the sample point that maximizes the average received energy. The early–late gate algorithm implements a discrete-time version of a continuous-time optimization to maximize a certain

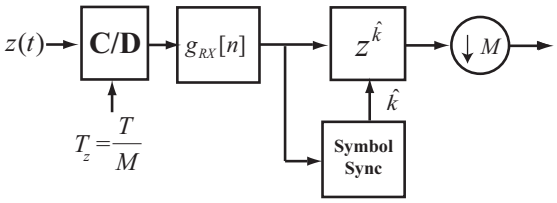


Figure 1: Receiver with symbol synchronization after the digital matched filtering.

cost function. For the best performance, both algorithms require that there is a lot of oversampling (i.e., M is large).

You will build two VIs, one for each of the two algorithms. After verifying the correctness of your design using a simulator, you will implement your code in lab using the NI-USRP hardware to see how your code works over a real wireless link. The goal of this lab is to understand the need for symbol timing recovery and the performance of two common symbol timing recovery algorithms.

In this lab, you will have to turn in two VIs (*align_MaxEnergy.vi* and *align_ELgate.vi*) and submit the answers to the questions in Section 2, along with the plot depicting timing error statistics, as part of the pre-lab submission.

1 Background

This section provides some background on the timing recovery problem and reviews the two algorithms that will be implemented in this lab. Remember that you will be implementing these synchronization methods using a purely digital approach.

1.1 Introduction to Symbol Timing Recovery

In the presence of frequency-flat fading as described in Eq. (1), the input to the receiver has the form

$$z(t) = \alpha e^{j\phi}\sqrt{E_x}\sum_m s[m]g_{\text{tx}}(t - mT - \tau_d) + v(t). \tag{2}$$

Now let $g(t)$ be the convolution of $g_{\text{tx}}(t)$ and $g_{\text{rx}}(t)$. With the model in Eq. (2) after matched filtering and sampling, the received signal is

$$y[n] = \sqrt{E_x}\alpha e^{j\phi}\sum_m s[m]g((n-m)T - \tau_d) + v[m].$$

Several different impairments are possible depending on the value of the τ_d. Unless otherwise specified, Nyquist pulse shapes are assumed.

First consider the case where τ_d is a fraction of a symbol period thus $0 < \tau_d < T$. This can model the effect of sample timing error, or not sampling at the right point. Under this assumption

$$y[n] = \underbrace{\sqrt{E_x}\alpha e^{j\phi}s[n]g(\tau_d)}_{\text{desired}} + \underbrace{\sqrt{E_x}\alpha e^{j\phi}\sum_{m\neq n} s[m]g((n-m)T - \tau_d)}_{\text{ISI}} + \underbrace{v[n]}_{\text{noise}}.$$

Intersymbol interference is created when the Nyquist pulse shape is not sampled exactly at nT. You will need *symbol synchronization* or *equalization* to mitigate the effect of sample timing error.

Second, suppose that $\tau_d = dT$ for some integer d. This is the case where symbol timing has been resolved but an unknown propagation delay, which is a multiple of the symbol period, remains. Under this assumption

$$\begin{aligned} y[n] &= \sqrt{E_x}\alpha e^{j\phi} \sum_m s[m]g((n-m)T - \tau_d) + v[m] \\ &= \sqrt{E_x}\alpha e^{j\phi} \sum_m s[m]g((n-m)T - dT) + v[m] \\ &= \sqrt{E_x}\alpha e^{j\phi} s[n-d] + v[m]. \end{aligned}$$

Essentially integer offsets create a mismatch between the indices of the transmitted and receive symbols. You will need *frame synchronization* to find the beginning of the data.

Third, suppose that the time error has been removed from the channel; equivalently $\tau_d = 0$. Then

$$y[n] = \sqrt{E_x}\alpha e^{j\phi} s[n] + v[m].$$

Sampling and removing delay leaves distortion due to α and ϕ, which are unknown to the receiver. The amplitude and phase shift will be estimated and equalized in Lab 4 (the focus of this lab is on the unknown delay τ_d).

1.2 The Maximum Output Energy Solution

Let $y(t)$ be the matched filtered version of $z(t)$ in continuous-time. Given an advance τ, notice that the output energy function defined as

$$\begin{aligned} J(\tau) = \mathbb{E}\,|y\,(nT+\tau)|^2 &= E_x \sum_m |g\,(mT + \tau - \tau_d)\,|^2 + \sigma_v^2 \\ &\leq E_x |g(0)|^2 + \sigma_v^2. \end{aligned}$$

The maximum of the function $J(\tau)$ occurs when $\tau - \tau_d$ is an integer multiple of the symbol rate. This value of τ is known as the maximum output energy solution. The resulting value of τ by that maximizes $J(\tau)$ is known as the maximum output energy solution. In this lab we implement two algorithms for finding a maximum output energy solution in discrete-time. Other criteria for performing symbol synchronization are possible, for example maximum likelihood solutions. The approach described here is robust to additive noise, certain kinds of fading channels, and small carrier frequency offsets.

You will implement a digital version of the maximization of $J(\tau)$. This means that you will perform the estimation and correction after the digital matched filter at the receiver, prior to downsampling.

1.3 Direct Maximization of the Output Energy in Discrete Time

Let

$$r[n] = \sum_m z[m] g_{\text{tx}}[n-m]$$

be the output of the matched receiver filter. The discrete-time output energy can be calculated as

$$J[k] = \mathbf{E}\left|r\left(nMT+k\right)\right|^2,$$

where k is the sample offset between $0, 1, \ldots, M-1$ corresponding to an estimate of $\hat{\tau} = kT/M$. The expectation can be replaced with a time average over P symbols to create the function

$$J_{\text{approx}}[k] = \frac{1}{P}\sum_{p=0}^{P-1}\left|r\left(pMT+k\right)\right|^2. \tag{3}$$

The direct maximum output energy solution is given by $\hat{k} = \max J_{\text{approx}}[k]$. Because the receiver is only performing symbol synchronization, it is sufficient to evaluate $J_{\text{approx}}[k]$ for values $k = 0, 1, \ldots, M-1$. The resulting value of $\hat{k}$ would be implemented in the advance operation in Figure 1 given in the Z-domain as $z^{\hat{k}}$. Instead of an advance, a delay could be used instead with value $z^{M-\hat{k}}$. Larger values of P generally give better performance.

1.4 Indirect Maximization of the Output Energy in Discrete Time

An alternative approach, useful for adaptive implementations, involves differentiating the cost function and discretizing the result. The approach here follows the description from [6]. Differentiating the expectation and assuming they can be interchanged

$$\begin{aligned}\frac{d}{d\tau}J_{opt}(\tau) &\simeq E\left\{\frac{d}{d\tau}\left|y\left(nT_s+\tau\right)\right|^2\right\}\\ &= E\left\{y(nT+\tau)\frac{\delta}{\delta\tau}y^*(nT+\tau)\right\}+E\left\{y^*(nT+\tau)\frac{\delta}{\delta\tau}y(nT+\tau)\right\}\\ &= 2\text{Re}\left[E\left\{y(nT+\tau)\frac{\delta}{\delta\tau}y^*(nT+\tau)\right\}\right].\end{aligned}$$

Using a first-order difference approximation

$$\frac{\delta}{\delta\tau}y^*(nT+\tau) \simeq y^*(nT_s+\tau+\delta) - y^*(nT_s+\tau-\delta)\,.$$

Now as before replacing the expectation with a time average

$$\frac{d}{d\tau}J_{opt}(\tau) \simeq \frac{1}{P}\sum_{n=1}^{P-1} 2\mathrm{Re}\left\{y(nT_s+\tau)\left(y^*(nT_s+\tau+\delta) - y^*(nT_s+\tau-\delta)\right)\right\}.$$

Choose δ to be a multiple of T/M. Using $r[n]$ as defined before let

$$J_\delta[k] = \sum_{n=0}^{P-1} 2\mathrm{Re}\left\{r[nP+k]\left(r^*[nP+k+\delta] - r^*[nP+k-\delta]\right)\right\}. \quad (4)$$

What we call the indirect maximum output energy solution is given by $\hat{k} = \max_{k=0,1,\ldots,M-1} J_\delta[k]$. A typical value of δ is 1.

2 Pre-Lab

Please answer the following questions about symbol timing recovery:

1. Show that in the absence of noise, α and ϕ in Eq. (2) do not have any impact on the maximum output energy solution.

2. What are the two critical assumptions used to formulate the indirect maximization of the output energy? Consider how the presence of the flat fading channel AWGN can impact this method. Specifically, using at least one of the critical assumptions, explain how you might mitigate the impact of these impairments by suitable selection of parameters.

3. After downsampling a sequence originally sampled at rate $1/T_z$ by a factor M, what is the sample period of the resulting signal?

Programming

In this pre-lab you will generate LabVIEW code to estimate the direct maximization of the output energy in discrete time and the early–late gate algorithm. Tables 1 and 2 describe the details of the VIs you must implement. Please name your VIs as per the convention *yourname_VIname.vi*.

Table 1: Description of *align_MaxEnergy.vi*

align_MaxEnergy.vi - compute sampling offset by maximizing $J_{\text{approx}}[k]$ and correct for offset			
INPUTS	*input complex waveform*	IQ Waveform cluster	received sequence after matched filtering
OUTPUTS	*output complex waveform*	IQ Waveform cluster	received sequence after symbol timing recovery
	offset	I32 (integer)	computed offset

Table 2: Description of *align_ELgate.vi*

align_ELgate.vi - estimate timing offset by minimizing $J_{\delta}[k]$ and correct for offset			
INPUTS	*input complex waveform*	IQ Waveform cluster	received sequence after matched filtering
OUTPUTS	*output complex waveform*	IQ Waveform cluster	received sequence after symbol timing recovery
	offset	I32 (integer)	computed offset

You have been provided with templates for the VIs you need to create for this lab that already have all the inputs and outputs wired for you. What is required of you is to finish constructing the block diagram to provide the functionality of the VIs.

Throughout the course, several of the VIs you will create will have a *modulation parameters* cluster passed into and out of them. The *modulation parameters in* contains many of the parameters needed by your VIs and will be unbundled from them. This has already been done for you in the template VIs if necessary. Some VIs will also have *modulation parameters out*, so that the cluster can be passed on through to the VIs that follow. Please ensure that these clusters remain wired the way they are in the template VIs, as changing how they are wired will cause VIs further down the line to break.

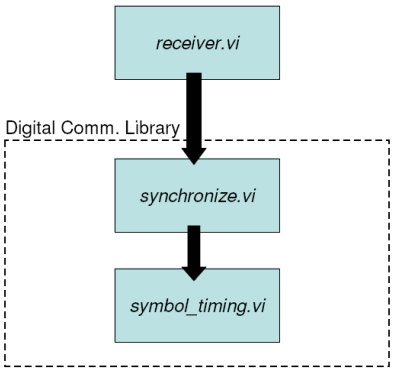


Figure 2: Hierarchy of code needed for symbol timing recovery in lab.

You will plug your code into the baseband simulator provided to you in order to verify functionality and correctness. You will not be replacing one of the main blocks inside *receiver.vi* as you have done in previous labs. Instead, you will need to insert your code into *symbol_timing.vi*, a subVI in the digital communications library built for this course. Figure 2 describes the hierarchy of the files needed for symbol timing recovery in this lab.

Figure 2 shows the block diagram of *synchronize.vi*, where you can find *symbol_timing.vi*. Notice there are a number of synchronization methods available in this VI, however you will only be dealing with *"Timing Estimation"* (the option for performing symbol timing recovery independent of frame synchronization or channel estimation).

After replacing *align_ELgate.vi* and *align_MaxEnergy.vi* in *symbol_timing.vi*, you should test your code by adding a small delay to your channel. Delay in the channel model can be modified using the *channel model parameters* control on the front panel of *Simulator.vi*. Note that delay in the channel model is bounded in *channel.vi* (i.e., you will not be able to add an arbitrarily large delay to the channel). You can use the front panel controls to modify the symbol timing recovery method used in the simulator. After choosing the STR method and modifying the delay, you should notice the estimated offset generated by the receiver changes as you vary delay. This integer offset value includes delay introduced by pulse shaping and matched filtering.

The simulator outputs the estimated delay of the channel (i.e., after correcting for any offset caused by filtering) as an error statistic. The VI *simulator.vi*

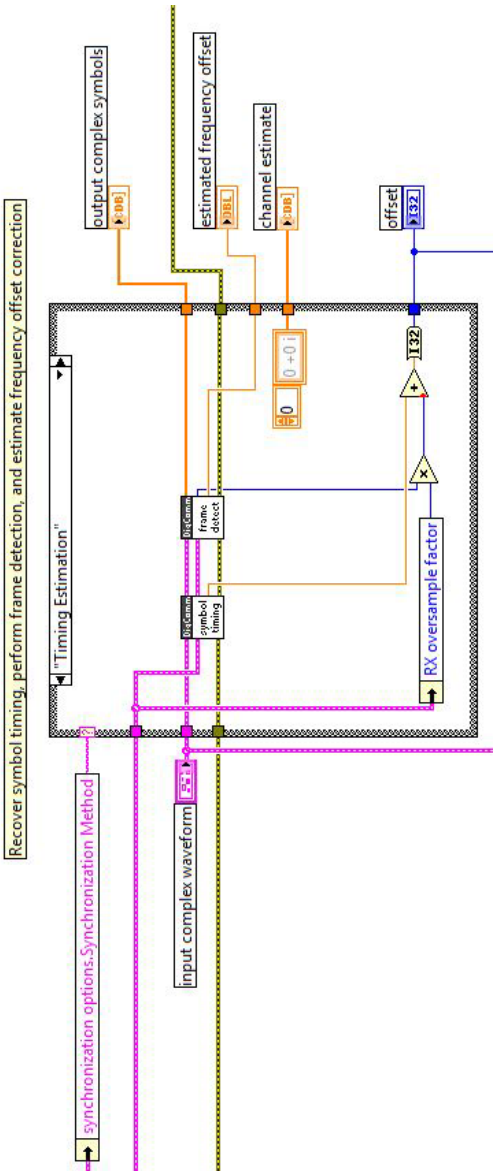


Figure 3: Block diagram of *synchronize.vi*.

compares this estimated delay with the actual delay and computes the mean square error static, which has been normalized by symbol period T_s

$$\epsilon[N] = E\left\{\left\|(\hat{\tau}(N) - \tau_d)/T_s\right\|^2\right\}, \tag{5}$$

where N is the oversample factor at the receiver, $\hat{\tau}(N)$ is the estimated delay of the channel computed at the receiver[1], and τ_d is the actual delay of the channel.

Plot a graph of the timing error statistic $\epsilon(N)$ versus oversample factor N, using the parameters below. Since your channel will have no noise, you will only need to run a single iteration of the simulator to gather the timing error statistic. Note when modifying oversample factor N, symbol period T_s must remain constant (i.e., since $T_s = NT_N$, you must also modify the sampling rate at the receiver appropriately).

- Delay $\tau_d = 0.34T_s$
- Oversample factor $N = 2, 4, 6, \ldots, 20$
- Channel Model: $h = 0.75e^{j\pi/4}$ (i.e., narrowband channel)
- Noise Power = $-\infty$ dB

In summary:

- After building *align_ELgate.vi* and *align_MaxEnergy.vi*, place these VIs into your simulator (i.e., inside *symbol_timing.vi*).
- Modify the symbol timing recovery method and the delay introduced by the channel from the front panel of *Simulator.vi*.
- After verifying the functionality of your code, observe how these algorithms perform in the presence of a narrowband channel with AWGN. In the presence of these impairments, observe how the packet length affects the performance of the algorithms you have implemented.
- Using delay $\tau_d = 0.34T_s$ and the parameters listed above, plot the timing error statistic $\epsilon(N)$, as defined in Eq. (5), for oversample factors $N = 2, 4, 6, \ldots, 20$ (i.e., even integers up to 20). Use a logarithmic scale for timing error and a linear scale for N.

[1] Estimated channel delay $\hat{\tau}(N) = \dfrac{\hat{k}T_s}{N}$, where $\hat{k}$ is the integer offset estimated at the receiver (after correcting for delay due to filtering), T_s is the symbol period, and N is the oversample factor at the receiver

Pre-Lab Turn In

1. Submit your implementation of *align_Elgate.vi* and *align_MaxEnergy.vi*.

   Note: If you need to generate additional subVIs for this or any future labs, please submit these subVIs along with your other pre-lab VIs.

2. Submit your answers to all of the questions in the pre-lab section along with your plot of timing error $\epsilon(N)$ versus oversample factor N.

3 Lab Experiment

In this lab you will run your implementations of *align_ELgate.vi* and *align_MaxEnergy.vi* over a real wireless link. Using your code from the prior labs in conjunction with the new blocks you have created, you will complete the framework for the transmitter and receiver blocks in lab. This framework will be provided to you as in previous labs.

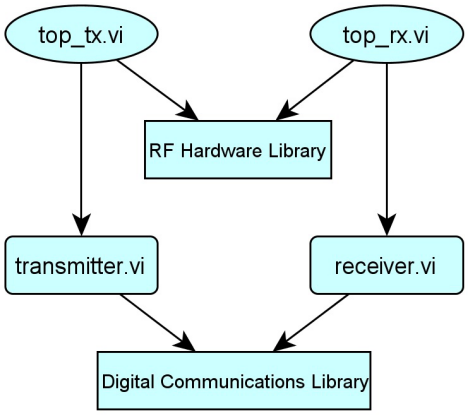


Figure 4: Hierarchy of code framework used in lab.

Insert your code into this framework as you did in the pre-lab. In the pre-lab you observed how timing error varies with the oversampling factor N at the receiver. Now you will observe this relationship over a real wireless link.

Set up the following parameters in your baseband system.

- Packet length = 500 bits
- Modulation type = QPSK

- Pulse shaping filter = Root Raised Cosine
- Filter parameter = 0.5
- Filter length = 8
- Channel model: $h = 0.75e^{j\pi/4}$ (i.e., narrowband channel)
- Noise power = $-$Inf dB
- TX sample rate = 20 MSamp/sec
- TX oversample factor = 20
- RX sample rate = 2 MSample/sec
- RX oversample factor = 2
- Symbol timing recovery method = Max Energy

Also make sure the following RF parameter is set up appropriately (using the HW parameters control accessible from the front panel of *top_rx.vi*):

- Capture time = 400 μsec

Questions

Answer the following questions about the digital communication system:

1. What is the symbol rate of the system based on the parameters above?

2. As you did in the pre-lab, vary the oversample factor N at the receiver. Observe the general shape of the received constellation for a number of packets (i.e., to eliminate any variance that might be caused by errors in frequency offset correction or channel estimation). Observe how the constellation changes when $N = 2, 4, 10$, and 20.

   Based on what you've observed and what you learned in the pre-lab, describe how the relationship between sampling error and oversample factor N manifests itself on the signal constellation of *top_rx.vi*.

3. For each of the oversample factors in the previous question (i.e., $N = 2, 4, 10, 20$), specify what value you set the *RX sample rate* to on the front panel of *top_rx.vi*.

Lab Turn In

Demonstrate to the instructor that your code is working properly. You will be required to identify that a sampling error has occurred and qualitatively support your claim based on the visualization tools available in *top_rx.vi*. You will then need to show the instructor how the performance of your system changes as you vary oversample factor N at the receiver. Also answer all of the questions above. You will be required to submit these answers in a lab report.

DIRECTIONS FOR LAB SUBMISSION

Your lab report should accomplish the following tasks.

1. Answer all questions from the lab experiment (i.e., do not reanswer pre-lab questions).
2. Discuss any problems you might have encountered and how you overcame these obstacles.
3. What underlying assumption about the noise in the system has been made, but not stated explicitly in this lab? Explain why this assumption is important for the maximum output energy maximization problem. *Hint: This is a property of AWGN*.
4. Air any grievances about the structure, content, or workload for this lab.